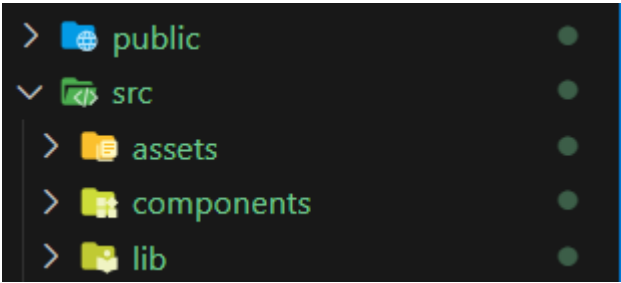




PP II - EXPLICAÇÃO DO PROJETO

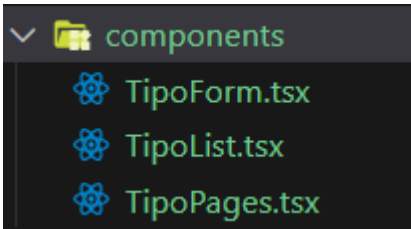
Semestre	4º Semestre
Tópicos Abordados	Algoritmos Códigos
Prioridade de Revisão	Média
Status de Compreensão	Parcialmente Compreendido
Matéria	UCE – Paradigmas de Programação II
Data da Aula	@28 de agosto de 2025
Professor	Daniel
Conteúdo	REACT.JS

Arquitetura do projeto



Components

Foram criado 3 componentes em React



TipoForm.tsx

```
import { useState } from "react";
import { api } from "../lib/api";
// IMPORTANDO VARIAIVEIS.

type Props = {
  onCreate: () => void;
  // QUANDO A FUNÇÃO onCreate FOR ACIONADA, ELA RETORNA NADA (void). APENAS IRÁ ATUALIZAR O FORM
};
// DECLARAÇÃO DE UMA PROPS

// DECLARANDO COMPONENT
export default function TipoForm({ onCreate }: Props) {
  // EXPORTANDO COMPONENTE COMO PADRÃO
  // PASSANDO A PROPS COMO PARAMETRO
  const [nome, setNome] = useState("");
  // DECLARANDO NOME COMO ESTADO PARA VAZIO
  const [loading, setLoading] = useState(false);
```

```

// DECLARANDO PROCESSO DE loading COMO ESTADO FALSO
const [erro, setErro] = useState<string | null>(null);
// DECLARANDO ERRO COMO UM ESTADO QUE PODE SER STRING OU NULL

// DECLARANDO FUNÇÃO DE MANIPULAÇÃO handle
async function handleSubmit(e: React.FormEvent) {
  // RECEBE UM FORMULARIO REACT
  e.preventDefault();
  // NÃO DEIXA O FORMULARIO RECARREGAR A PAGINA
  setErro(null);
  // ERRO NULO

  const trimmed = nome.trim();
  // RECEBE O NOME PASSADO PELO FORMULARIO E TIRA TODOS OS ESPAÇOS ANTES E DEPOIS
  // EXEMPLO DE .trim()
  // const nome = "  Diego Melo  ";
  // console.log(nome.trim());
  // "Diego Melo"

  if (!trimmed) {
    // SE O NOME ESTIVER VAZIO
    setErro("Informe um nome.");
    // ERRO INFORMADO
    return;
  }
  setLoading(true);
  // MUDOU loading PARA VERDADEIRO
  try {
    // COMEÇO DO BLOCO try...catch
    // CHAMA FUNÇÃO COM await DA API PARA CADASTRAR NO BANCO USANDO O NOME COMO PARAMETRO
    await api.criarTipo(trimmed);
    // APÓS O NOME CADASTRADO NO BANCO
    setNome("");
    // MUDOU O NOME DO FORMULARIO PARA UMA STRING VAZIA
    onCreate();
    // ATUALIZOU O FORMULARIO USANDO PROPS
  } catch (err: any) {
    // SE ALGUM ERRO FOR CRIADO
    setErro(err?.message ?? "Erro ao criar tipo");
    // CRIARÁ ESSE NOVO ERRO, E NADA DO BLOCO try SERÁ EXECUTADO
  } finally {
    // CASO O BLOCO catch FOR EXECUTADO, O BLOCO finally SERÁ EXECUTADO, CANCELANDO O LOADING
    setLoading(false);
  }
}
// FINALIZOU BLOCO DE MANIPULAÇÃO
}
// COMEÇO DO COMPONENTE
return (
  // CRIAÇÃO DE UM FORMULARIO POR TAG HTML
  <form onSubmit={handleSubmit} className="card space-y-3">
    <!--
    onSubmit É UM METODO HTML QUE É EXECUTADO SEMPRE QUE UM FORMULARIO FOR ENVIADO

    →
    <div>
    <!--
    CONFIGURAÇÃO DO INPUT E LABEL
    →

```

```

    <label className="label">Nome do tipo</label>
    <input
      className="input"
      placeholder="ex.: Deficiência Cognitiva"
      value={nome}
      onChange={(e) ⇒ setNome(e.target.value)}
      disabled={loading}
    />
    {erro && <p className="error">{erro}</p>}
  </div>
  <div className="flex justify-end">
    <button disabled={loading} className="btn btn-primary">
      {loading ? "Salvando..." : "Criar tipo"}
    </button>
  </div>
</form>
);
// FIM DO COMPONENTE
}

```

TipoList.tsx

```

import type { TipoDeficiencia } from "../types";
// IMPORTANDO VARIÁVEIS

// DECLARANDO PROPS
type Props = {
  tipos: TipoDeficiencia[];
  // PROPS RESGATA UMA TIPAGEM DO BD E CRIA UM ARRAY COM ESSE TIPO
};

// DECLARANDO COMPONENTE
export default function TipoList({ tipos }: Props) {
  if (!tipos.length) {
    // SE O ARRAY ESTIVER VAZIO
    return <div className="card text-gray-500">Nenhum tipo cadastrado ainda.</div>;
  }
  return (
    // COMPONENTE CASO ARRAY ESTIVER AO MENOS 1 REGISTRO
    <div className="card">
      <h3 className="text-lg font-semibold mb-3">Tipos cadastrados</h3>
      <ul className="divide-y">
        <!--
        FUNÇÃO .map() DO JS PARA PERCORRER O ARRAY E MOSTRAR DADOS
        →
        {tipos.map((t) ⇒ (
          <li key={t.id} className="py-2 flex items-center justify-between">
            <div>
              <span className="font-medium">{t.nome}</span>
            </div>
            <span className="text-xs text-gray-400">#{t.id}</span>
          </li>
        ))}
      </ul>
    </div>
  );
}

```

```
// FIM DO COMPONENTE
}
```

TipoPages.tsx

```
import { useEffect, useState } from "react";
import TipoForm from "../components/TipoForm";
import TipoList from "../components/TipoList";
import { api } from "../lib/api";
import type { TipoDeficiencia } from "../types";
// IMPORTANDO VARIÁVEIS

// DECLARANDO COMPONENTE
export default function TiposPage() {
  const [tipos, setTipos] = useState<TipoDeficiencia[]>([]);
  // DECLARANDO TIPOS COMO ARRAY DE TIPOS COMO ESTADO VAZIO
  const [loading, setLoading] = useState(true);
  // DECLARANDO PROCESSO DE loading COMO ESTADO FALSO
  const [erro, setErro] = useState<string | null>(null);
  // DECLARANDO ERRO COMO UM ESTADO QUE PODE SER STRING OU NULL

  async function carregar() {
    // DECLARANDO ERRO COMO NULO E loading COMO TRUE
    setErro(null);
    setLoading(true);
    try {
      // COMEÇO DO BLOCO try...catch
      // CHAMA FUNÇÃO DA API DE LISTAR TIPOS
      const data = await api.listarTipos();
      // O RESULTADO DA FUNÇÃO ESTÁ DENTRO DA VARIÁVEL data
      setTipos(data);
      // MUDOU O VALOR DA PROPS PARA O VALOR DA FUNÇÃO
    } catch (err: any) {
      // SE ALGUMA COISA FALHAR NO BLOCO try... O BLOCO catch SERÁ EXECUTADO
      setErro(err?.message ?? "Erro ao carregar tipos");
      // CRIARÁ ESSE NOVO ERRO, E NADA DO BLOCO try SERÁ EXECUTADO
    } finally {
      // CASO O BLOCO catch FOR EXECUTADO, O BLOCO finally SERÁ EXECUTADO, CANCELANDO O LOADING
      setLoading(false);
    }
  }

  // HOOK DO REACT QUE EXECUTA EFEITOS COLATERAIS EM COMPONENTES FUNCIONAIS
  useEffect(() => {
    // AQUI CHAMAMOS A FUNÇÃO carregar(), QUE PODE BUSCAR DADOS OU INICIALIZAR ALGO
    carregar();
    // O ARRAY DE DEPENDÊNCIAS VAZIO [] SIGNIFICA QUE ESSE EFEITO
    // SERÁ EXECUTADO APENAS UMA VEZ, QUANDO O COMPONENTE FOR MONTADO
  }, []);

  // COMEÇO DO COMPONENTE
  return (
    <div className="container-page space-y-6 py-8">
      <header>
        <h1 className="text-2xl font-bold">Tipos de Deficiência</h1>
        <p className="text-gray-600">Crie e consulte os tipos cadastrados na API.</p>
      </header>
```

```

    <TipoForm onCreate={carregar} />

    {loading ? (
      <div className="card">Carregando...</div>
    ) : erro ? (
      <div className="card text-red-600">{erro}</div>
    ) : (
      <TipoList tipos={tipos} />
    )}
  </div>
);
// FIM DO COMPONENTE
}

```

api.tsx

```

import type { TipoDeficiencia } from "../types";
// IMPORTANDO VARIÁVEIS

const BASE_URL = import.meta.env.VITE_API_URL ?? "http://localhost:3000";
// RESGATA PORTA HTTPS DO BACK-END PELO .env

// FUNÇÃO HTTP GENÉRICA QUE USA FETCH PARA FAZER REQUISIÇÕES
// <T> INDICA QUE O RETORNO PODE SER QUALQUER TIPO DEFINIDO PELO CHAMADOR
async function http<T>(path: string, init?: RequestInit): Promise<T> {
  // REALIZA A REQUISIÇÃO USANDO FETCH, CONCATENANDO A URL BASE COM O PATH
  const res = await fetch(`${BASE_URL}${path}`, {
    // DEFINE O HEADER "CONTENT-TYPE" COMO JSON
    // E JUNTA COM OUTROS HEADERS QUE POSSAM VIR DE INIT
    headers: { "Content-Type": "application/json", ...(init?.headers || {}) },
    // ESPALHA (MERGE) AS OUTRAS CONFIGURAÇÕES RECEBIDAS EM INIT (METHOD, BODY, ETC.)
    ...init,
  });

  // SE A RESPOSTA NÃO FOR OK (STATUS FORA DE 200–299), TRATA O ERRO
  if (!res.ok) {
    // TENTA LER A RESPOSTA COMO TEXTO
    const text = await res.text().catch(() => "");
    // DEFINE UMA MENSAGEM DE ERRO PADRÃO
    let msg = text || res.statusText || "ERRO NA REQUISIÇÃO";
    try {
      // TENTA INTERPRETAR O TEXTO COMO JSON
      const j = JSON.parse(text);
      // SE EXISTIR UM CAMPO "error" NO JSON, USA ELE COMO MENSAGEM
      msg = j.error || msg;
    } catch {
      // SE NÃO DER PARA INTERPRETAR COMO JSON, LANÇA O ERRO COM A MENSAGEM DEFINIDA
      throw new Error(msg);
    }
  }

  // RETORNA A RESPOSTA COMO JSON, TIPADA COMO T
  return res.json() as Promise<T>;
}

// OBJETO API QUE EXPÕE FUNÇÕES ESPECÍFICAS USANDO A FUNÇÃO HTTP GENÉRICA

```

```
export const api = {
  // FAZ UM GET EM /tipos E RETORNA UMA LISTA DE TipoDeficiencia[]
  listarTipos() {
    return http<TipoDeficiencia[]>("/tipos");
  },

  // FAZ UM POST EM /tipos, ENVIANDO { nome } NO CORPO, E RETORNA UM TipoDeficiencia
  criarTipo(nome: string) {
    return http<TipoDeficiencia>("/tipos", {
      method: "POST",
      body: JSON.stringify({ nome }),
    });
  },
};
```